

MODULE :- Automation with Selenium

EXPERIMENT TITLE :- Windows, Tabs, and Iframes (Assignment 6)

1.Problem Statement :-

To automate a web application containing an **iframe and a new browser tab** using Selenium WebDriver. The task is to switch between the main page and iframe, interact with elements inside the iframe, open and switch to a new browser tab, retrieve its title and URL, close the new tab, and return to the main window.

2. Tools and Software Required

S.	Tool/Software	Purpose
1	Python	Programming language for automation
2	VS Code	IDE for writing and executing the Selenium script
3	Google Chrome	Web browser used for testing
4	Selenium WebDriver	Automates browser actions
5	ChromeDriver	Connects Selenium with Chrome
6	WebDriverWait	Waits for required elements/windows
7	Test Automation Practice Website	Used for new-tab automation
8	The Internet – Iframe Page	Used for iframe automation

3. Concepts Used

- Selenium WebDriver
- **Iframes**
- `driver.switch_to.frame()`
- `driver.switch_to.default_content()`
- **Browser Windows/Tabs**
- `driver.window_handles`
- `driver.current_window_handle`
- `driver.switch_to.window()`
- `driver.close()`
- `WebDriverWait`
- Expected Conditions
- Element identification using ID and XPath
- JavaScript execution using `execute_script()`

4.Source Code :-

File 1: assignment_6.py

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time

driver = webdriver.Chrome()
driver.maximize_window()

try:

    # =====
    # PART 1: IFRAME
    # =====

    print("\n=====")
    print("PART 1: IFRAME")
    print("=====")

    driver.get("https://the-internet.herokuapp.com/iframe")

    print("Iframe practice page opened")
    time.sleep(3)
```

```

# Locate iframe
iframe = WebDriverWait(driver, 10).until(
    EC.presence_of_element_located(
        (By.ID, "mce_0_ifr")
    )
)

# Switch into iframe
driver.switch_to.frame(iframe)

print("Successfully switched into iframe")

time.sleep(2)

# Locate editor inside iframe
editor = WebDriverWait(driver, 10).until(
    EC.presence_of_element_located(
        (By.ID, "tinymce")
    )
)

print("Text inside iframe:", editor.text)

# -----
# DO NOT use editor.click()
# It causes ElementClickInterceptedException.
# Focus the editor using JavaScript instead.
# -----

driver.execute_script(
    "arguments[0].focus();",
    editor
)

# Select existing text
editor.send_keys(" - Selenium Python")

print("Text entered successfully inside iframe")

time.sleep(3)

# Switch back to main document
driver.switch_to.default_content()

print("Successfully switched back to main page")

time.sleep(2)

# =====
# PART 2: NEW TAB
# =====

print("\n=====")
print("PART 2: NEW TAB")
print("=====")

```

```
driver.get(
    "https://testautomationpractice.blogspot.com/"
)

print("Test Automation Practice page opened")

time.sleep(3)

# Store original window
main_window = driver.current_window_handle

print("Main window:", main_window)

# Find New Tab button
new_tab_button = WebDriverWait(driver, 10).until(
    EC.element_to_be_clickable(
        (
            By.XPATH,
            "//button[contains(normalize-space(), 'New Tab')]"
        )
    )
)

# Click New Tab
new_tab_button.click()

print("New Tab button clicked")

# Wait for second window/tab
WebDriverWait(driver, 10).until(
    EC.number_of_windows_to_be(2)
)

# Get all window handles
windows = driver.window_handles

print("Number of windows:", len(windows))

# Switch to new tab
for window in windows:
    if window != main_window:
        driver.switch_to.window(window)
        break

print("Successfully switched to new tab")

time.sleep(3)

# Print new tab information
print("New tab title:", driver.title)
print("New tab URL:", driver.current_url)

time.sleep(3)
```

```
# Close new tab
driver.close()

print("New tab closed")

# Switch back to original window
driver.switch_to.window(main_window)

print("Successfully switched back to main window")

time.sleep(3)

print("\n=====")
print("PASS: ASSIGNMENT 6 COMPLETED")
print("=====")

finally:

    driver.quit()
```

5.Implementation Steps :-

- 1.Install Python and Selenium WebDriver.
- 2.Open the iframe practice page using Selenium.
- 3.Locate the iframe using its ID.
- 4.Switch from the main page to the iframe using `driver.switch_to.frame()`.
- 5.Locate the editable text area inside the iframe.
- 6.Read the existing text and enter additional text.
- 7.Switch back to the main page using `driver.switch_to.default_content()`.
- 8.Open the Test Automation Practice website.
- 9.Locate and click the **New Tab** button.
- 10.Store the main window handle using `driver.current_window_handle`.
- 11.Use `driver.window_handles` to identify the newly opened tab.
- 12.Switch to the new tab using `driver.switch_to.window()`.
- 13.Retrieve and display the new tab's title and URL.
- 14.Close the new tab using `driver.close()`.
- 15.Switch back to the original/main window.
- 16.Verify that both iframe and new-tab operations were completed successfully.
- 17.Close the browser.

6.Output :-

The screenshot displays two browser windows. The top window, titled 'Automation Testing Practice', shows a website with a bokeh background. It features a navigation bar with links: Home, Udemy Courses, Online Trainings, Blog, and PlaywrightPractice. The main content area is divided into two sections: 'Data Entry Form' on the left and 'Tabs' on the right. The form includes input fields for Name, Email, Phone, and Address. The 'Tabs' section shows a search bar with the letter 'W' and a 'Dynamic Button' labeled 'START'. Below this is an 'Alerts & Popups' section. The bottom window, titled 'The Internet', shows a page with an iFrame containing a TinyMCE WYSIWYG Editor. The editor's toolbar is visible, and a message box indicates that the editor is in read-only mode due to a limit on editor loads. A green banner on the right side of the page says 'Post me on GitHub'. The footer of the page mentions 'Powered by Elemental Selenium'.

Automation Testing Practice

For Selenium, Cypress & Playwright

Home Udemy Courses Online Trainings Blog PlaywrightPractice

Data Entry Form

Name:

Email:

Phone:

Address:

Tabs

W

Dynamic Button

Alerts & Popups

An iFrame containing the TinyMCE WYSIWYG Editor

File Edit View Format

Paragraph B I

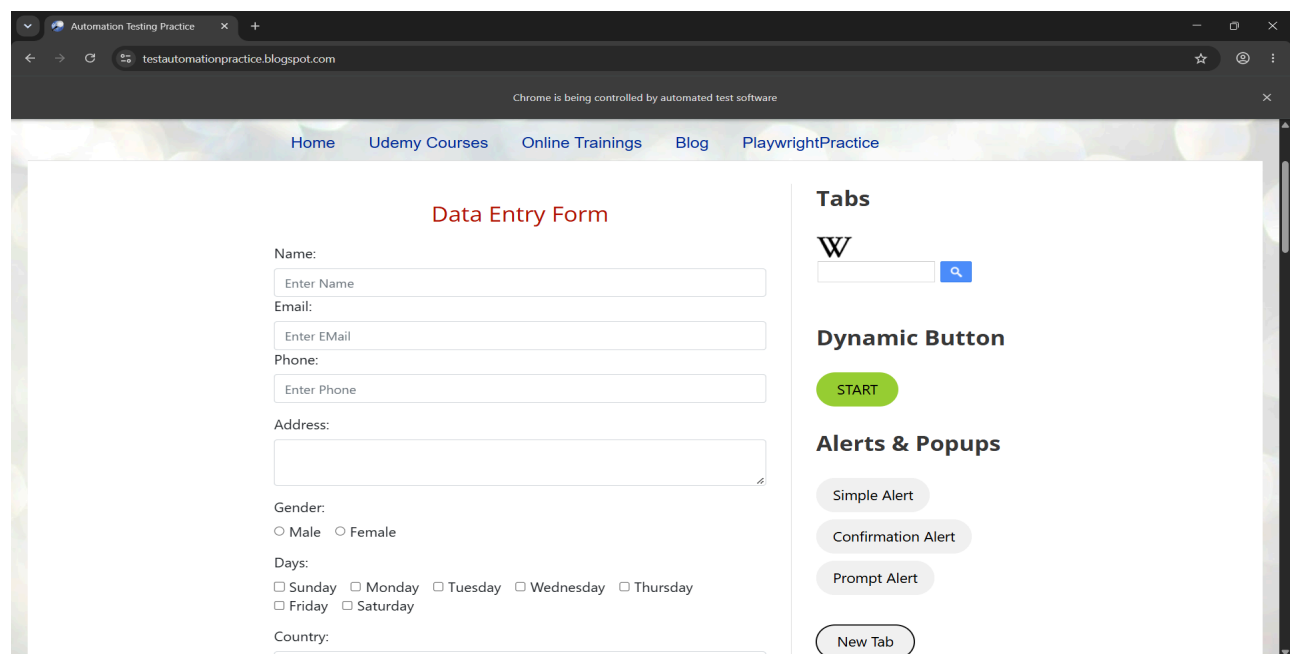
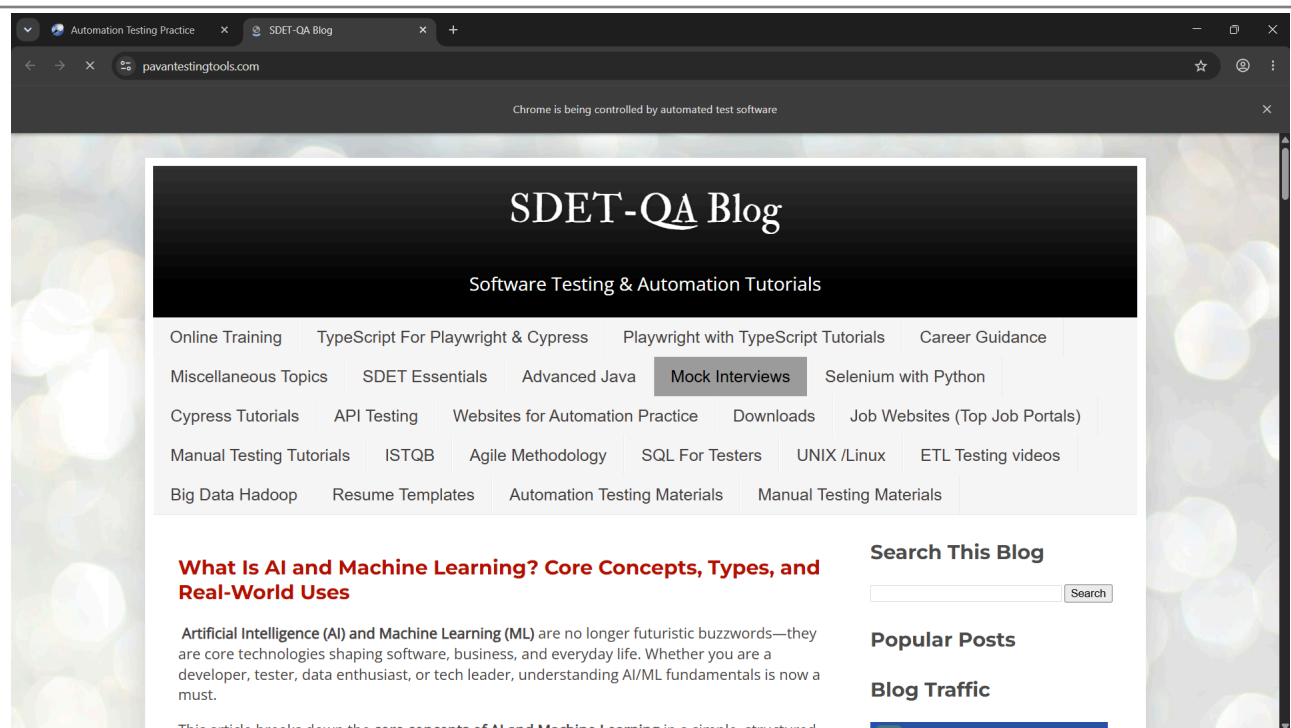
Your content goes here

TinyMCE is in read-only mode because you have no more editor loads available this month. Please request that the admin upgrade your plan or add a valid payment method for additional editor load charges. [Learn more.](#)

POWERED BY TINY

Powered by [Elemental Selenium](#)

Post me on GitHub



7.Result :-

The Selenium script successfully:

- Switched into the iframe and interacted with its content.
- Switched back to the main webpage.
- Opened a new browser tab.
- Switched from the main window to the new tab.
- Retrieved the new tab's title and URL.
- Closed the new tab.
- Switched back to the main browser window.

8.Observation :-

The experiment demonstrated that Selenium provides specific methods for handling multiple browsing contexts. `switch_to.frame()` is required to interact with elements inside an iframe, while `window_handles` and `switch_to.window()` are used to manage multiple browser tabs or windows.

9.Conclusion :-

Selenium WebDriver was successfully used to automate **iframes and multiple browser tabs/windows**. The experiment provided practical understanding of switching between different browsing contexts and interacting with elements located inside an iframe and a newly opened browser tab.